C 语言关键字 extern、函数指针与指针函数、指针数组与数组指针笔记整理

一、C 语言关键字 extern

1. 概念与作用

- extern 是 C 语言中的一个**关键字**，用于声明变量或函数在其他文件或作用域中定义，告诉编译器“这个标识符的定义在别处”。
- **声明外部变量**：用于在一个文件中引用另一个文件定义的全局变量。
- **声明外部函数**：用于在一个文件中引用另一个文件定义的函数（函数默认就是 extern）。

2. 用法举例

2.1 外部变量声明 假设有两个文件：

file1.c

```c
int count = 10; // 定义全局变量
```

file2.c

```c
extern int count; // 声明外部变量
void foo() {
    printf("%d\n", count); // 可以直接使用
}
```

2.2 外部函数声明 **file1.c**

```c
void hello() {
    printf("Hello!\n");
}
```

file2.c

```c
extern void hello(); // 声明外部函数（其实可以省略 extern）
int main() {
    hello(); // 可以直接调用
    return 0;
}
```

3. 特点与注意事项

- *extern* 只是**声明**，不分配内存，不初始化。
- 变量或函数必须在其他地方有**定义**，否则链接时报错。
- 常用于**头文件**，声明外部变量或函数接口。
- 不能用 *extern* 初始化变量（如 *extern int a = 5;* 是错误的）。
- 如果只声明不定义，链接时会报“未定义符号”错误。

4. 常见场景

- 多文件程序共享全局变量。
- 头文件声明外部变量或函数，源文件定义。

5. 关于局部变量与 *extern*

- *extern* 只能用于**声明外部的全局变量或函数**，不能用于声明局部变量。
- 局部变量的作用域仅限于当前函数或代码块，不能被其他文件或函数访问。
- *extern* 的本意是“声明在其他地方定义的全局变量或函数”，局部变量不具备这种特性。
- **结论：**
  extern 不能声明局部变量，只能声明全局变量或函数。

6. 总结

- *extern* 用于声明变量或函数在其他文件或作用域中定义，方便多文件间共享和调用。
- 只声明不定义，不分配空间、不初始化。
- 不能用于声明局部变量，只能声明全局变量或函数。

二、C 语言中的函数指针与指针函数

1. 函数指针

1.1 概念

- **函数指针**是指向函数的指针变量，可以用来保存函数的地址，并通过该指针调用函数。

1.2 声明方式

返回类型 *(*指针变量名*)(*参数类型列表*);*

例如：

```
int (*fp)(int, int); // fp 是指向返回 int、参数为两个 int 的函数的指针
```

1.3 用法举例

```
int add(int a, int b) { return a + b; }
int main() {
    int (*fp)(int, int) = add; // 让 fp 指向 add 函数
    int result = fp(2, 3);     // 通过函数指针调用函数
    printf("%d\n", result);    // 输出 5
    return 0;
}
```

1.4 主要用途

- 回调函数
- 实现多态
- 数组存储多个函数指针（如菜单、事件处理）

---

2. 指针函数

2.1 概念

- **指针函数**是指返回值类型为指针的函数，即函数的返回值是一个指针。

2.2 声明方式

返回类型* 函数名(参数类型列表);

例如：

```
int* get_pointer(int* p) { return p; }
```

2.3 用法举例

```
int a = 10;
int* get_pointer(int* p) { return p; }
int main() {
    int* pa = get_pointer(&a); // pa 指向 a
    printf("%d\n", *pa);       // 输出 10
    return 0;
}
```

2.4 主要用途

- 返回数组、结构体、变量的地址
- 动态内存分配相关函数（如 *malloc*）

---

3. 比较与区别

名称	概念	声明方式	作用
函数指针	指向函数的指针变量	返回类型 (* 变量名)(参数)	保存函数地址，调用函数
指针函数	返回值为指针的函数	返回类型 * 函数名(参数)	返回指针

- **函数指针**是变量，保存函数地址，可用于回调和动态调用。
- **指针函数**是函数，返回一个指针值，常用于返回地址或动态分配内存。
- 两者本质不同：一个是“指针”，一个是“函数”。

4. 总结

- **函数指针**：指向函数的指针变量，可通过它调用函数。
- **指针函数**：返回值类型为指针的函数。
- 区别在于：函数指针是变量，指针函数是函数，二者用途和声明方式不同。

三、指针数组与数组指针

1. 指针数组（*Array of Pointers*）

1.1 概念

- 指针数组是**数组**，数组中的每个元素都是**指针**（通常指向同一类型的数据）。

1.2 声明方式

```
int *arr[5];
```

含义：*arr* 是一个**包含 *5* 个元素的数组**，每个元素都是 *int* * 类型的指针。

1.3 使用场景

- 常用于存放字符串指针（如 `char *argv[]`）
- 存放多个变量的地址

1.4 示例

```c
int a = 1, b = 2, c = 3;
int *arr[3] = {&a, &b, &c};
printf("%d\n", *arr[0]); // 输出 1
```

---

2. 数组指针（Pointer to Array）

2.1 概念

- 数组指针是**指向数组的指针**，它本身是一个指针，指向一个数组的首地址。

2.2 声明方式

```c
int (*p)[5];
```

含义：`p` 是一个指针，**指向包含 5 个 `int` 元素的数组**。

2.3 使用场景

- 需要传递整个二维数组的一行给函数时
- 动态操作多维数组

2.4 示例

```c
int arr[5] = {1, 2, 3, 4, 5};
int (*p)[5] = &arr;
printf("%d\n", (*p)[0]); // 输出 1
```

---

3. 区别与对比

	指针数组	数组指针
本质	数组，元素为指针	指针，指向一个数组
声明方式	`int *arr[5];`	`int (*p)[5];`
内存结构	连续存放多个指针	存放一个指向数组的指针
访问方式	`arr[i]` 得到指针	`(*p)[i]` 访问数组元素

	指针数组	数组指针
常见用途	字符串数组、指针集合	操作多维数组、数组传参

例子对比

```c
// 指针数组
char *strs[3] = {"hello", "world", "c"};
printf("%s\n", strs[1]); // 输出 world

// 数组指针
int arr[5] = {1,2,3,4,5};
int (*p)[5] = &arr;
printf("%d\n", (*p)[2]); // 输出 3
```

4. 总结

- **指针数组**：本质是“数组”，每个元素是指针，常用于存放多个地址（如字符串数组）。
- **数组指针**：本质是“指针”，指向一个数组，常用于多维数组操作和数组整体传参。
- 两者在声明、用途和内存结构上有本质区别，需根据实际需求选择使用。

四、复杂声明的顺指针螺旋移动法

1. 方法说明

- 以变量名为中心，顺时针旋转，遇到中括号、小括号或者星号时，将其翻译成对应的名称，最后将所得到的名称序列进行组合、润色，就是我们声明语句的解释。

2. 例子

```c
int* (*arr[3])(int)
```

- *arr* 是包含三个元素的数组，元素是指针，指针的类型是参数类型为 *int* 的函数，这个函数的返回值是 *int**。
- 即这是一个**指针函数数组**，数组的元素类型为指针函数（返回值为 *int** 的函数）：数组里面存放着 *3* 个指针函数。

五、strlen("\0") 结果说明

- strlen 计算的是**字符串中第一个 \0（结束符）之前的字符数量**。
- "\0" 是一个只包含结束符的字符串，没有其他字符。
- 所以 strlen("\0") 返回 0，**不会计算结束符本身的长度**。

总结：
strlen 只计算结束符之前的字符数，不包括结束符本身。

六、复杂类型 sizeof 运算解析

1. 代码片段

```
int* (*arr)[4];
```

2. 类型解析

- arr 是一个**指向包含 4 个 int 指针的数组的指针**。
- 换句话说，arr 的类型是“指向 int* 数组（长度为 4）的指针”。

3. sizeof 运算区别

表达式	类型	含义	sizeof 结果
arr	int* (*)[4]	指向 4 个 int* 数组的指针	指针大小（如 8 字节）
*arr	int* [4]	4 个 int* 数组	4× 指针大小（如 32 字节）
arr[4]	int* (*)[4]	指向下一个 4 个 int* 数组的指针	指针大小（如 8 字节）

4. 联系

- arr 是指针，指向一个 int* 数组（长度为 4）。
- *arr 是这个数组本身，可以访问 (*arr)[0] ~ (*arr)[3]。
- arr[4] 是指针运算，表示“跳过 4 个数组”，类型仍然是 int* (*)[4]，但通常不会这样用。

如需进一步举例或代码演示，请继续提问！